Homework 2 - Writeup

Team members

Tania Ortiz-Rosales

Soniya Pradeepkumar Phaltane

1. [4 pts] Which of the following components of program state are shared across threads in a multithreaded process?

   a. Register values b. Heap memory c. Global variables d. Stack memory

   Answer:

   - The program states that are shared across threads in multithreaded processes are the **(b)Heap memory** and (c)**Global variables**

2. [8 pts]The pseudocode of the figure below illustrates the basic push() and pop() operations of an array-based stack. Assuming that this algorithm could be used in a concurrent environment, answer the following questions:

   (a) What data have a race condition?

   (b) How could the race condition be fixed?

   Answer: (A) - In the given pseudocode, the data that have a race condition are:

   1. top variable:
      - The top variable keeps track of the index of the next available position for push() and the current position for pop(). It is shared between threads, and when multiple threads attempt to perform push() or pop() operations simultaneously, there could be interference in updating the top value, leading to inconsistent results. For example, two threads might simultaneously read the top value, increment it, and overwrite each other's changes, causing incorrect updates.
   2. Stack [] array:
      - The stack array is also shared. When multiple threads are accessing the same index of the array to either push or pop items, there's a risk that one thread might overwrite the data of another thread or read the wrong value.

(B) Race condition be fixed by:

1. The race condition can be fixed by including mutex into the code. This would allow critical selection as a block of code. Therefore, it means that the specific thread that is executing must be completed and return either the current number of items or the item itself depending on the function called. In essence, it would prevent the intervening of the threads in the concurrent environments and ensure that the correct output is always returned by the hardware.
2. Another option is to use semaphores to control access to the push() and pop() operations. A semaphore can be used to signal when a thread can safely access the stack, ensuring that the operations are serialized and race conditions are avoided

3. [4 pts] What is the difference between concurrent execution and parallel execution?

Answer:

	Concurrent execution	Parallel execution
1.	Multiple tasks are started, but only one is executed at a time.	Multiple tasks are executed at the same time, with each task running on a separate processor or core.
2.	In concurrent execution, tasks are broken down into smaller time slices, but they share a single processor. Each task runs for a brief period, and then the processor switches to another task. This allows multiple tasks to make progress at the same time, but they don't run simultaneously.	In parallel execution, a large task is divided into smaller, independent subtasks. These subtasks are executed simultaneously on different processors or cores. This division of work reduces the overall time to complete a task by leveraging multiple processors working at the same time.

4. [5 pts] Servers can be designed to limit the number of open connections. For example, a server may wish to have only socket connections at any point in time. As soon as connections are made, the server will not accept another incoming connection until an existing connection is released. Illustrate how semaphores can be used by a server to limit the number of concurrent connections.

Answer:

Semaphores can be used by the server to limit the number of concurrent connections by initializing the semaphore to the max amount of open connections that are allowed within the server. Once the semaphore value eventually hits 0, then all other connections will be ignored until the next open connection slot is available. By following this pattern, it prevents the server from becoming overloaded and allows easy management of all the connections.

Example:

Initial Semaphore Value (N=3):

- Total Semaphores: 3 (slots available)
  1. Connection 1: wait() → Semaphore: 2
  2. Connection 2: wait() → Semaphore: 1
  3. Connection 3: wait() → Semaphore: 0
  4. Connection 4: (rejected, no slots)
  5. Connection 1 released: signal() → Semaphore: 1 (new connection accepted)

Pseudocode:

```
Semaphore availableConnections = N                    // Max concurrent connections
accept_connection() {
      if (availableConnections > 0) {
            wait(availableConnections)                // Reserve a slot
            handle_connection()                       // Process the connection
      }
else {
            reject_connection()                       // Reject new connection
      }
}
connection_released() {
   signal(availableConnections)                       // Increment semaphore
   close_connection()
}
```

5. [9 pts] Assume that a system has multiple processing cores. For each of the following scenarios, describe which is a better locking mechanism—a spinlock or a mutex lock where waiting processes sleep while waiting for the lock to become available:
   (a) The lock is to be held for a short duration.
   (b) The lock is to be held for a long duration.
   (c) A thread may be put to sleep while holding the lock.

   Answer:

   - (A) - In the case of a lock being held for a short duration, the spinlock would be the best choice because since the duration is so short, it removes the unnecessary actions of putting every thread to sleep, which is what is needed for a mutex lock. Therefore, the spin lock would save time in comparison to the mutex lock
   - (B) - In the case of a lock being held for a long period of time, the best case scenario would be the mutex lock. The mutex lock is the best scenario because it would eliminate the excessive CPU wait time that is required in the spin locks. Therefore, by using the mutex lock, the CPU would be saving cycles by placing all the other threads to sleep.
   - (C) - In the case of the thread being put to sleep while holding the lock, the best case scenario would be a mutex lock. This lock would be the best scenario as all the waiting threads would just waste the CPU otherwise.

6. [5 pts] Describe how the signal() operation associated with monitors differs from the corresponding operation defined for semaphores.

   Answer:

   Monitors: signal() Operation

   - The signal() operation in monitors is used to notify a waiting thread that a specific condition has been met, such as a resource becoming available or a required condition being satisfied.
   - Monitors combine mutual exclusion and synchronization, allowing only one thread to access the monitor at a time. When signal() is called, it wakes up exactly one waiting thread from the queue.

Semaphores: signal() Operation

- The signal() operation in semaphores increments the semaphore's value, indicating that a resource is available or the semaphore is released, which may unblock a waiting thread.
- Semaphores don't manage mutual exclusion or condition synchronization. They require manual management of synchronization across threads.
- When signal() is called, it either unblocks one waiting thread or increments the semaphore's value if no threads are waiting. It helps in managing resource availability across multiple threads.

7. [5 pts] The Linux kernel has a policy that a process cannot hold a spinlock while attempting to acquire a semaphore. Explain why this policy is in place.

   Answer:

   - The Linux kernel has this policy in place that a process cannot hold a spinlock while attempting to acquire a semaphore because it prevents deadlocks from occurring.
   - A **spinlock** keeps a thread busy-waiting until it acquires the lock, while a **semaphore** may cause a thread to block if the resource is unavailable. If a thread holds a spinlock and tries to acquire a semaphore, it could end up in a situation where it's waiting for the semaphore, but the semaphore is waiting for the spinlock, causing a deadlock.
   - Additionally, holding a spinlock while blocking on a semaphore would waste CPU cycles because the thread would keep spinning while waiting for a resource, which could have been used by other threads.
   - Therefore, this policy prevents that and also improves CPU efficiency.